

Solventa

Aseguradora digital sobre Finanzas Abiertas y Datos Abiertos

Dirección de Arquitectura

Documento de Arquitectura de Software

Diseño de experimentos

Versión 1.0

Documento técnico

Equipo de arquitectura

Integrante	Rol
Jazmin Natalia Córdoba Puerto	Gerencia de proyecto, usabilidad y entrega
Juan Esteban Mejía Izasa	Web front, integración de APIs y pagos
Miguel Alejandro Gómez Alarcón	Arquitectura, Open Finance, KYC, rendimiento y seguridad
Angie Natalia Arandio Niño	Dominio, web back, móvil y pruebas

Bogotá D.C. · 6 de septiembre de 2026

Experimento 1 — Escalamiento de 500 a 50.000 cotizaciones por minuto

Título del experimento	Experimento 1. Escalamiento de 500 a 50.000 cotizaciones por minuto
Propósito del experimento	Validar que el recorrido de cotización embebida escale 100 veces, desde 500 hasta 50.000 cotizaciones por minuto, sin incumplir la latencia ni perder aislamiento entre socios.
Resultados esperados	Cumple si procesa 50.000 cotizaciones/min, mantiene $p95 \leq 250$ ms y $p99 \leq 500$ ms, habilita capacidad adicional en ≤ 60 s, conserva una tasa de error $< 1\%$ y ningún socio supera 1,2 veces la mediana del p95. Se refuta si falla cualquiera de esos umbrales de volumen, latencia, error o equidad.
Recursos requeridos	Apache JMeter 5.6 distribuido; 50 socios y credenciales sintéticas; perfiles de prueba; WireMock para dependencias externas; despliegue EKS con HPA (parámetros de escalamiento horizontal); PostgreSQL 15; Redis 7; CloudWatch, X-Ray y trazas por salto.
Elementos de arquitectura involucrados	Escenario 6.2 Campaña masiva de socio y ASR-1.1. Vistas funcional y de despliegue. Componentes: API Gateway, Cotización y Rating, Perfilamiento y Scoring, Redis, PostgreSQL y Adaptador Open Finance. Sensibilidades: autoescalado, caché, cuotas por socio y depósitos de conexiones.
Esfuerzo estimado	16 horas-persona: preparación 6 h, ejecución 4 h y análisis 6 h; tres corridas completas.

Hipótesis de diseño del experimento 1	
<i>Si Cotización y Perfilamiento escalan horizontalmente, usan perfiles precalculados en Redis y aíslan recursos por socio y dependencia, el sistema sostendrá 50.000 cotizaciones/min sin exceder p95 de 250 ms ni p99 de 500 ms.</i>	
Punto de sensibilidad	Umbral de CPU del HPA del 70 % y ventana de 30 s; capacidad mínima; tasa de acierto y vencimiento del caché; tamaño de los depósitos HTTP, Redis y PostgreSQL; cuotas y límite de tasa por socio.
Historia de arquitectura asociada	SOL-43 — HU-ARQ-09 Autoescalado horizontal.
Nivel de incertidumbre	Alta. La arquitectura documenta los mecanismos, pero no existe evidencia de que sus umbrales respondan a un salto de 100 veces ni de que Redis o PostgreSQL dejen de ser cuellos de botella.

Patrones de arquitectura asociados al experimento 1	Descripción
P1 Caché de perfiles con lectura anticipada	Sirve perfiles vigentes desde Redis y precalcula los de campañas programadas para sacar Open Finance del camino crítico.
P4 Mamparo de aislamiento	Reserva depósitos separados por dependencia y aplica cuotas por socio para que una saturación no consuma todos los recursos.
P12 Autoescalado con comprobaciones de salud	Añade réplicas sanas de Cotización y Perfilamiento ante el aumento de CPU y retira instancias no disponibles.
Microservicios por contexto de dominio	Permite escalar Cotización y Perfilamiento sin replicar todos los servicios del núcleo.

Tácticas de arquitectura asociadas al experimento 1	Descripción
Replicación horizontal	Aumentar las réplicas de los servicios del camino de cotización cuando crece la demanda.
Reducir demanda computacional	Usar caché y precálculo para evitar llamadas externas y cálculos repetidos durante el pico.
Controlar y aislar recursos	Aplicar límites de tasa por socio y depósitos separados para HTTP, Redis y PostgreSQL.
Detectar saturación	Medir CPU, colas, conexiones, aciertos de caché, tasa de error y latencia por percentil y por socio.

Listado de componentes involucrados en el experimento 1		
Componente o microservicio	Propósito y comportamiento esperado	Tecnología asociada
API Gateway	Autentica, aplica cuotas por socio y distribuye las solicitudes sin favorecer un origen.	Gateway, WAF, TLS 1.3
Cotización y Rating	Compone la prima y sostiene el volumen objetivo sin exceder su presupuesto de procesamiento.	Python 3.11, Flask, Gunicorn
Perfilamiento y Scoring	Obtiene el perfil desde caché o la fuente controlada y escala de forma independiente.	Python 3.11, Flask, Gunicorn
Redis	Responde perfiles vigentes con alta tasa de acierto y sin saturar conexiones.	Redis 7, redis-py
PostgreSQL	Mantiene datos transaccionales y absorbe consultas sin agotar el depósito.	PostgreSQL 15, psycopg
Adaptador Open Finance	Traduce la integración y respeta el tiempo de espera definido; el proveedor se simula estable.	httpx, WireMock

Listado de conectores involucrados en el experimento 1		
Conector	Comportamiento deseado en el experimento	Tecnología asociada
Socio → API Gateway	Aceptar tráfico de 50 socios con autenticación, cuota y medición independiente.	HTTPS REST, TLS 1.3
API Gateway → Cotización	Enrutar 50.000 solicitudes/min sin introducir un cuello de botella.	HTTP persistente
Cotización → Perfilamiento	Mantener el salto dentro del presupuesto de latencia aun durante el escalamiento.	HTTP/1.1, httpx
Perfilamiento → Redis	Resolver aciertos de caché con latencia estable y depósitos separados.	RESP, redis-py
Perfilamiento → PostgreSQL	Usar el depósito transaccional sin bloquear la ruta online.	SQL, psycopg
Perfilamiento → Adaptador Open Finance	Respetar timeout y no amplificar el pico hacia el proveedor.	HTTP, httpx
Adaptador → Proveedor simulado	Entregar respuestas deterministas para aislar la capacidad interna.	HTTPS REST, WireMock

Tecnología asociada con el experimento Desarrollo infraestructura y almacenamiento	Justificación
Lenguajes de programación: Python 3.11	Implementa los servicios y permite instrumentar métricas por salto y por socio.
Plataforma de despliegue: Docker Compose y AWS EKS con HPA	Reproduce el empaquetado del proyecto y permite medir el autoescalado horizontal en ≤ 60 s.
Bases de datos: PostgreSQL 15 y Redis 7	Representan la persistencia transaccional y la caché que pueden limitar el volumen.
Herramientas de análisis: JMeter 5.6, CloudWatch, X-Ray y WireMock	Generan los escalones, estabilizan el proveedor y miden throughput, percentiles, errores y réplicas.
Librerías: httpx, redis-py, psycopg y pybreaker	Ejercitan conexiones persistentes, depósitos y políticas de resiliencia reales del diseño.
Frameworks de desarrollo: Flask y Unicorn	Ejecutan los servicios backend con múltiples trabajadores y configuración medible.

Distribución de actividades por integrante del experimento 1		
Integrante	Tareas a realizar	Esfuerzo estimado
Jazmin Natalia Córdoba Puerto	Coordinar la preparación, registrar riesgos y consolidar la evidencia y el acta de resultados.	3 h
Juan Esteban Mejía Izasa	Preparar los 50 socios sintéticos, el tráfico por API y los escenarios de WireMock.	4 h
Miguel Alejandro Gómez Alarcón	Configurar carga, HPA, métricas de rendimiento y analizar cuellos de botella.	5 h
Angie Natalia Arandio Niño	Preparar datos, ejecutar las tres corridas y conciliar errores y respuestas.	4 h

Experimento 2 — Carga online, procesamiento batch y eventos simultáneos

Título del experimento	Experimento 2. Carga online, procesamiento batch y eventos simultáneos
Propósito del experimento	Validar el aislamiento entre tráfico online, reprocesamiento masivo y procesamiento de eventos, incluida la contrapresión cuando un consumidor se atrasa.
Resultados esperados	Cumple si 25.000 cotizaciones/min conservan $p95 \leq 250$ ms y $p99 \leq 500$ ms; 10 millones de perfiles terminan en < 2 h; se reciben 1 millón de eventos en ≤ 10 min sin pérdida; tras pausar un consumidor 2 min, el lag vuelve a cero en ≤ 15 min; y existen cero efectos de negocio duplicados. Se refuta ante pérdida, duplicidad, degradación online, batch fuera de plazo o lag que no converge.
Recursos requeridos	Apache JMeter 5.6; 10 millones de perfiles sintéticos; generador de 1 millón de eventos; Kafka 3.x o Amazon MSK; ejecutor batch; EKS con límites por carga; Redis 7; PostgreSQL 15; CloudWatch, X-Ray y medición de lag y offsets.
Elementos de arquitectura involucrados	Escenarios 6.2 Evento paramétrico masivo y Reprocesamiento de perfiles, junto con Cotización embebida. ASR-1.1 y ASR-3.1. Vistas funcional y de despliegue. Componentes: Cotización, Perfilamiento, Kafka, consumidores, Redis y PostgreSQL. Sensibilidades: particiones, lotes, cuotas, depósitos y contrapresión.
Esfuerzo estimado	20 horas-persona: preparación 8 h, ejecución 6 h y análisis 6 h; tres corridas completas.

Hipótesis de diseño del experimento 2	
<i>Si las cargas usan depósitos y cuotas separados, Kafka absorbe el pico con contrapresión y los consumidores procesan de forma idempotente, el online conservará su latencia mientras batch y eventos cumplen sus metas de volumen y tiempo.</i>	
Punto de sensibilidad	Número de particiones y consumidores; tamaño de lote; pausas del consumidor; cuotas de CPU y memoria; depósitos de PostgreSQL y Redis; tasa de acierto de caché; umbral de backpressure y velocidad de drenaje.
Historia de arquitectura asociada	SOL-37 — HU-ARQ-08 Calibración CB Open Finance.
Nivel de incertidumbre	Alta. Las cargas compiten por CPU, memoria, conexiones y almacenamiento; una configuración válida por separado puede fallar al superponerse o acumular un lag que no se drena.

Patrones de arquitectura asociados al experimento 2	Descripción
P8 Publicación y suscripción de eventos de dominio	Kafka desacopla productores y consumidores, retiene eventos y permite relectura después de una pausa.
P4 Mamparo de aislamiento	Separa cuotas, trabajadores y depósitos para que batch o eventos no agoten los recursos del online.
P1 Caché de perfiles con lectura anticipada	Protege la cotización online y reduce competencia por PostgreSQL durante el reproceso.
P12 Autoescalado con comprobaciones de salud	Aumenta capacidad de servicios y consumidores sin mantener instancias no saludables en rotación.

Tácticas de arquitectura asociadas al experimento 2	Descripción
Aplicar contrapresión	Reducir o pausar admisión cuando el consumidor supera el lag permitido, sin descartar eventos.
Aislar recursos	Asignar cuotas y depósitos separados a online, batch y consumidores.
Particionar y paralelizar	Distribuir eventos y perfiles por clave estable para aumentar throughput sin romper el orden requerido.
Retener y reintentar	Confirmar offsets solo después del efecto persistido y releer fallos de forma idempotente.
Supervisar colas y capacidad	Medir lag, velocidad de drenaje, tiempos batch, saturación y percentiles online durante toda la superposición.

Listado de componentes involucrados en el experimento 2		
Componente o microservicio	Propósito y comportamiento esperado	Tecnología asociada
API Gateway	Mantiene la admisión y las cuotas del tráfico online durante la prueba combinada.	Gateway, WAF, TLS 1.3
Cotización y Rating	Procesa 25.000 cotizaciones/min dentro de los percentiles objetivo.	Python 3.11, Flask, Gunicorn
Perfilamiento y Scoring online	Responde desde perfiles vigentes y conserva su presupuesto de latencia.	Python 3.11, Redis 7
Worker batch de Perfilamiento	Reprocesa 10 millones de perfiles en < 2 h con recursos acotados.	Python 3.11, contenedor dedicado
Kafka o Amazon MSK	Acepta 1 millón de eventos, conserva offsets y expone el lag por partición.	Kafka 3.x, KRaft o MSK
Siniestros y consumidores de eventos	Procesan eventos paramétricos sin pérdidas ni efectos duplicados.	Python 3.11, kafka-python
Redis	Sirve el online y recibe actualizaciones del batch sin agotar conexiones.	Redis 7, redis-py
PostgreSQL	Persiste resultados y efectos de consumidores con depósitos separados.	PostgreSQL 15, psycopg

Listado de conectores involucrados en el experimento 2		
Conector	Comportamiento deseado en el experimento	Tecnología asociada
API Gateway → Cotización	Sostener la carga online sin degradación durante batch y eventos.	HTTP persistente
Cotización → Perfilamiento	Mantener p95 y p99 aun cuando el perfilamiento batch esté activo.	HTTP/1.1, httpx
Perfilamiento online → Redis	Priorizar lecturas online y mantener conexiones disponibles.	RESP, redis-py
Worker batch → PostgreSQL	Leer perfiles por lotes sin monopolizar el depósito transaccional.	SQL por lotes, psycopg
Worker batch → Redis	Actualizar perfiles con tasa limitada para no expulsar ni bloquear lecturas online.	RESP, redis-py
Productores → Kafka	Publicar 1 millón de eventos en ≤ 10 min sin rechazo ni pérdida.	Kafka, kafka-python
Kafka → Consumidores	Aplicar contrapresión durante la pausa y drenar el lag en ≤ 15 min.	Consumer groups y offsets
Consumidores → PostgreSQL	Persistir cada efecto una sola vez antes de confirmar el offset.	SQL transaccional, psycopg

Tecnología asociada con el experimento Desarrollo infraestructura y almacenamiento	Justificación
Lenguajes de programación: Python 3.11	Implementa servicios, worker batch, productores y consumidores instrumentados.
Plataforma de despliegue: Docker Compose y AWS EKS	Permite separar contenedores, cuotas y autoescalado por tipo de carga.
Bases de datos: PostgreSQL 15, Redis 7 y Kafka 3.x o MSK	Cubren persistencia, caché y retención de eventos, los tres recursos compartidos que se deben observar.
Herramientas de análisis: JMeter 5.6, CloudWatch, X-Ray y métricas de lag	Correlacionan percentiles online, duración batch, offsets, pérdidas, duplicados y saturación.
Librerías: kafka-python, redis-py, psycopg y httpx	Ejecutan publicación, consumo, caché, persistencia y llamadas internas con depósitos reales.
Frameworks de desarrollo: Flask y Gunicorn	Ejecutan la ruta online y permiten controlar trabajadores separados del batch.

Distribución de actividades por integrante del experimento 2		
Integrante	Tareas a realizar	Esfuerzo estimado
Jazmin Natalia Córdoba Puerto	Coordinar la ventana de prueba, registrar riesgos y consolidar la interpretación y la evidencia.	4 h
Juan Esteban Mejía Izasa	Preparar el generador de carga online y la publicación del millón de eventos.	4 h
Miguel Alejandro Gómez Alarcón	Configurar particiones, cuotas, contrapresión, observabilidad y análisis de capacidad.	6 h
Angie Natalia Arandio Niño	Preparar los 10 millones de perfiles, ejecutar el batch y conciliar pérdidas y duplicados.	6 h

Experimento 3 — Reintentos y resultados ambiguos en pagos

Título del experimento	Experimento 3. Reintentos y resultados ambiguos en pagos
Propósito del experimento	Validar que un cobro o desembolso produzca un solo efecto financiero cuando la pasarela procesa la operación, pero la respuesta se pierde, llega tarde o provoca reintentos concurrentes.
Resultados esperados	Cumple si, en 10.000 operaciones lógicas, existen cero cobros o desembolsos duplicados, cero transacciones confirmadas perdidas, el 100 % de reintentos con la misma clave recupera el mismo resultado y toda operación ambigua converge a confirmada o rechazada en ≤ 60 s. Se refuta ante un solo duplicado, una confirmación perdida, estados divergentes o una operación que no converge.
Recursos requeridos	Apache JMeter 5.6; claves de idempotencia y datos sintéticos; WireMock como pasarela; Toxiproxy para cortes después del procesamiento; PostgreSQL 15; Kafka o MSK para conciliación y auditoría; CloudWatch, X-Ray, trazas y consultas de conciliación.
Elementos de arquitectura involucrados	Escenario 6.3 Recaudo y pago de indemnizaciones y ASR-2.1. Vistas componente-conector e información. Componentes: Suscripción y Emisión, Pagos y Recaudo, Adaptador de Pasarela, PostgreSQL, Kafka y Auditoría. Interfaces: CobrarPrima y PasarelaPago. Sensibilidades: atomicidad, clave, timeout y conciliación.
Esfuerzo estimado	18 horas-persona: preparación 6 h, ejecución 5 h y análisis 7 h; tres corridas completas.

Hipótesis de diseño del experimento 3

Si Pagos registra atómicamente la clave de idempotencia y el estado del intento, reintentando solo fallos transitorios y reconcilia respuestas ambiguas consultando la pasarela, cada operación lógica producirá exactamente un efecto financiero.

Punto de sensibilidad	Alcance y vencimiento de la clave; restricción única; transición de estados; instante del commit frente a la llamada externa; timeout; número y separación de reintentos; consulta de estado y frecuencia de conciliación.
Historia de arquitectura asociada	SOL-114 — HU-ARQ-12 Idempotencia y conciliación de pagos ante reintentos.
Nivel de incertidumbre	Alta. Un timeout no permite saber si la pasarela ejecutó la operación; un reintento ingenuo puede duplicar dinero y una falla entre la llamada y el commit puede dejar estados inconsistentes.

Patrones de arquitectura asociados al experimento 3	Descripción
P5 Puertos y adaptadores	El núcleo usa CobrarPrima y PasarelaPago sin depender del contrato particular de una pasarela.
P3 Tiempo de espera con reintento acotado	Limita la espera y reintenta solo operaciones idempotentes con espera creciente y componente aleatorio.
Receptor idempotente por clave de negocio	Registra una clave única y devuelve el mismo resultado ante repeticiones de la misma operación lógica.
P8 Publicación y suscripción	Publica confirmaciones y tareas de conciliación sin duplicar el efecto financiero sincrónico.

Tácticas de arquitectura asociadas al experimento 3	Descripción
Prevenir duplicados	Aplicar restricción única por clave y una máquina de estados monotónica para aceptar una sola transición terminal.
Reintentar con seguridad	Reintentar únicamente timeouts y errores transitorios, con límite, espera creciente y jitter.
Recuperar resultado ambiguo	Consultar el estado externo antes de emitir una segunda solicitud y conciliar operaciones pendientes.
Mantener atomicidad local	Persistir clave, intento, correlación externa y estado dentro de una transacción PostgreSQL.
Registrar y correlacionar	Trazar solicitud lógica, intento externo, evento, conciliación y efecto financiero con un identificador común.

Listado de componentes involucrados en el experimento 3		
Componente o microservicio	Propósito y comportamiento esperado	Tecnología asociada
Suscripción y Emisión	Solicita el cobro de la prima una sola vez por operación de negocio.	Python 3.11, Flask
Pagos y Recaudo	Controla idempotencia, estados, reintentos y conciliación; produce un solo efecto financiero.	Python 3.11, Flask, Unicorn
Adaptador de Pasarela	Traduce autorizar, confirmar, consultar y reversar al contrato del proveedor.	Python 3.11, httpx
Pasarela externa simulada	Procesa éxito o rechazo y permite respuestas tardías, perdidas y consultas de estado.	WireMock, Toxiproxy
PostgreSQL	Persiste claves, intentos, correlaciones y estado final con restricción única.	PostgreSQL 15, psycopg
Kafka o Amazon MSK	Transporta confirmaciones, conciliación y auditoría sin modificar el cobro sincrónico.	Kafka 3.x, kafka-python
Consentimientos y Auditoría	Registra la secuencia completa y permite conciliar el efecto financiero.	Python 3.11, PostgreSQL

Listado de conectores involucrados en el experimento 3		
Conector	Comportamiento deseado en el experimento	Tecnología asociada
Suscripción → Pagos	Enviar una clave estable y recuperar el mismo resultado ante reintentos.	Interfaz CobrarPrima, HTTP
Pagos → PostgreSQL	Bloquear o reutilizar la clave y persistir cada transición de forma atómica.	SQL transaccional, psycopg
Pagos → Adaptador de Pasarela	Invocar el puerto sin conocer detalles del proveedor y sin duplicar solicitudes en curso.	Interfaz PasarelaPago
Adaptador → Pasarela simulada	Propagar la clave; simular éxito, timeout, respuesta tardía y conexión cortada.	HTTPS REST, httpx, Toxiproxy
Conciliador de Pagos → Pasarela	Consultar el estado externo antes de reintentar una operación ambigua.	HTTPS REST, httpx
Pagos → Kafka	Publicar el estado confirmado con el mismo identificador lógico.	Kafka, kafka-python
Kafka → Auditoría	Consumir relecturas sin registrar dos efectos para la misma operación.	Consumer group y clave de evento

Tecnología asociada con el experimento Desarrollo infraestructura y almacenamiento	Justificación
Lenguajes de programación: Python 3.11	Implementa servicio de pagos, adaptador, conciliador e instrumentación de estados.
Plataforma de despliegue: Docker Compose y AWS EKS	Aísla servicio, simulador, base y broker; permite repetir fallos de red de forma controlada.
Bases de datos: PostgreSQL 15 y Kafka 3.x o MSK	PostgreSQL garantiza unicidad y transacciones; Kafka conserva eventos de conciliación y auditoría.
Herramientas de análisis: JMeter 5.6, WireMock, Toxiproxy, CloudWatch y X-Ray	Generan concurrencia, inducen ambigüedad y correlacionan operaciones, reintentos y efectos.
Librerías: httpx, psycopg, kafka-python y pytest	Ejercitan integración HTTP, persistencia transaccional, eventos y verificaciones automatizadas.
Frameworks de desarrollo: Flask y Unicorn	Ejecutan el servicio y permiten validar reintentos concurrentes en múltiples trabajadores.

Distribución de actividades por integrante del experimento 3		
Integrante	Tareas a realizar	Esfuerzo estimado
Jazmin Natalia Córdoba Puerto	Coordinar escenarios, registrar riesgos y consolidar la interpretación y el informe.	3 h
Juan Esteban Mejía Izasa	Configurar el adaptador, WireMock, claves de idempotencia y escenarios de la pasarela.	6 h
Miguel Alejandro Gómez Alarcón	Diseñar fallos con Toxiproxy, trazas, criterios financieros y análisis de ambigüedad.	5 h
Angie Natalia Arandio Niño	Preparar datos, ejecutar concurrencia y conciliar claves, estados y efectos finales.	4 h

